Smart Face Attendance Monitoring System Using OpenCV

Computer Vision Laboratory Project - Documentation

Student name(s): _____ Course/Section: _____ Date: October 2026

1. Title and 2. Objectives

Title: Smart Face Attendance Monitoring System Using OpenCV.

General objective: to build a Python application that processes a live webcam feed, recognises registered students or employees and records their attendance automatically without duplicate entries.

- Detect faces in real time using a Haar Cascade classifier.
- Apply image operations (resize, BGR to RGB, grayscale, blur) to prepare frames.
- Recognise registered people and display name, confidence, date, time and status on screen.
- Save Name, Date, Time and Status to Attendance.csv, marking each person once per day.
- Add motion detection as a bonus to reduce unnecessary processing.

3. Problem Statement

Manual roll calls and signature sheets consume class time, are easy to falsify (a friend can sign for an absent person) and produce paper records that are slow to analyse. A contactless system is needed that identifies people automatically, records the time of arrival and guarantees that the same person is not logged twice on the same day.

4. Use Case

A webcam is placed at the classroom or office entrance. People are registered once (about 20 photos each). During the session the program runs continuously: when a registered person steps in front of the camera, the box turns green, the status shows *Present* and one new row is added to Attendance.csv. A teacher or HR officer then opens the CSV in Excel or pandas to review attendance. Typical settings: small classrooms, laboratories, small offices and events.

5. Technologies Used

Technology	Purpose
Python 3	Main programming language
OpenCV (cv2)	Webcam capture, Haar Cascade detection, image operations, drawing
face_recognition (dlib)	Pre-trained deep model producing 128-number face encodings and face distances
NumPy	Array and distance computations
pandas, csv	Reading and writing Attendance.csv, checking duplicates
datetime, os, pickle, time	Date/time stamps, file handling, storing encodings, FPS

6. Techniques Used

6.1 Haar Cascade Classifier

A Haar Cascade is a classical machine-learning detector. It evaluates thousands of simple rectangular brightness features (for example, the eye region is darker than the cheeks) on a grayscale image. The features are combined by AdaBoost into stages; most non-face windows are rejected by the first cheap stages, which makes detection fast enough for live video. In the project `detectMultiScale` is called with scaleFactor = 1.1, minNeighbors = 5 and minSize = 30x30, using the file haarcascade_frontalface_default.xml. The detected boxes (x, y, w, h) are passed to the recognition step so the heavy model does not have to search the whole frame again.

6.2 Image Operations

- **Resize:** frames are shrunk to 50% before detection, about four times fewer pixels and a clear FPS gain; boxes are scaled back to full size.

- **BGR to RGB:** OpenCV stores colour as BGR but face_recognition expects RGB, so cv2.cvtColor is applied before encoding.
- **Grayscale:** Haar Cascades and motion detection work on one brightness channel.
- **Blur and equalisation:** a Gaussian blur removes sensor noise and histogram equalisation improves contrast in dim rooms; a larger blur is used in motion detection so small noise is ignored.

6.3 Image Analysis and Transformation

- **Bounding boxes** drawn around each face; colour shows state (orange = verifying, green = present, red = unknown).
- **Labels:** person's name with confidence score, plus the attendance status text under the box.
- **Translucent overlay:** a header panel created with cv2.addWeighted shows title, date, current time, registered count, people present today, FPS and motion status.
- **Analysis:** variance of the Laplacian measures image sharpness so blurry registration photos are rejected; frame differencing (absdiff, threshold, dilate) measures the percentage of moving pixels (bonus motion detection).

6.4 System workflow

capture_faces.py saves cropped face photos -> train_faces.py converts them to encodings (models/encodings.pickle) -> main.py reads each frame, detects faces (Haar), encodes them, finds the closest registered encoding (distance <= 0.5), requires 3 consecutive confirmations and then writes one CSV row if the person is not already present today.

7. Screenshots

[Insert screenshot: Figure 1 - Face registration (capture_faces.py)]

[Insert screenshot: Figure 2 - Training output (train_faces.py)]

[Insert screenshot: Figure 3 - Live recognition with green box and Present status]

[Insert screenshot: Figure 4 - Unknown face (red box)]

[Insert screenshot: Figure 5 - Attendance.csv opened in Excel]

[Insert screenshot: Figure 6 - Motion status IDLE / ACTIVE]

8. Discussion

The two-stage design is the key idea: the lightweight Haar Cascade locates faces quickly, and the deep model identifies only those faces. Processing every second frame, downscaling for detection and motion gating keep the system usable on an ordinary laptop CPU. The 3-frame confirmation rule trades a short delay (a fraction of a second) for far fewer wrong entries. The duplicate rule is enforced by checking Attendance.csv itself, so it survives restarts. The tolerance value (0.5) is a trade-off: a lower value rejects look-alikes but may reject the right person in poor light, a higher value does the opposite.

9. Accuracy

The recognition model used by the face_recognition library reports 99.38% accuracy on the Labeled Faces in the Wild (LFW) benchmark. This is a laboratory figure on still photos; **accuracy of this project on a live webcam depends on lighting, camera quality, face angle and the quality of the registration photos and must be measured, not assumed.** Suggested test: ask each registered person to approach the camera 20 times and ask 5 unregistered people to do the same, then fill in the table below.

Test condition	Attempts	Correct	Wrong person	Unknown / missed	Accuracy (%)
Good front lighting, no accessories					
Dim lighting					
Head turned about 30 degrees					
With glasses					
Unregistered people (should be Unknown)					

Formula: Accuracy = (correct identifications / total attempts) x 100. Report your measured values in the Discussion or Conclusion when presenting.

10. Limitations

- Haar Cascades detect mainly frontal, upright faces and give occasional false detections on textured backgrounds.
- Performance drops in very dark or strongly back-lit scenes, and with masks or sunglasses.
- No liveness detection: a printed photo or phone screen could be accepted (spoofing).
- CPU-only processing limits FPS when several faces are present.
- CSV storage is simple but not suitable for large organisations or concurrent users.
- Face data is personal data: consent and secure storage are required.

11. Future Improvements

- Replace Haar with a DNN or MediaPipe detector for side-view robustness; add MediaPipe facial landmarks.
- Liveness detection (blink or head-movement challenge) to stop photo spoofing.
- Store records in a database (SQLite/MySQL) and build a web dashboard with daily and monthly reports.
- Time-in / time-out and late-arrival status; email or SMS notifications.
- GPU acceleration and a nearest-neighbour index for large numbers of people; encrypt stored encodings.

12. Conclusion

The project delivers a complete, modular live attendance system built with Python and OpenCV. It combines a Haar Cascade classifier, image operations, image analysis and transformation, and a bonus motion detector with deep face encodings to identify registered people and log each of them once per day in Attendance.csv. The code is organised into separate scripts for registration, training, recognition and utilities, which makes it easy to understand, test and extend. With careful registration and good lighting it provides a practical, low-cost alternative to manual attendance, and the limitations listed above define clear directions for future work.